Homework 09 — STAT 432 (Fall 2026)

Name: Owen Park **NetID:** owenp3

Everything is in Python (numpy 1.26, pandas 2.2, scikit-learn 1.6). The one random seed in Question 1 is **4329**, passed to `numpy.random.default_rng` to assign the five folds. `famhist` is coded 1 for `Absent` and 2 for `Present`, and the other eight predictors are used as they are.

Question 1: Choosing an SVM kernel

I assign each of the 462 observations to one of five folds once and reuse those folds for every candidate. Inside each fold the scaler is fit on the four training folds only, through a pipeline, so the held-out fold never touches the standardization. The polynomial kernel is $(\gamma\, x^{\mathsf{T}} x' + r)^d$, and for all three degrees I hold $\gamma = 1/9$ (one over the number of predictors, which is scikit-learn's `"scale"` value for standardized data) and $r = 1$ fixed, so only C moves. With $r = 1$ the degree-d kernel contains every lower-degree term as well. The radial kernel is $\exp(-\gamma \lVert x - x' \rVert^2)$, and there I vary both C and γ. That gives 19 candidates: 4 linear, 3 for each polynomial degree and 6 radial.

In [1]:
```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
pd.set_option("display.width", 130)

heart = pd.read_csv("data/saheart.csv")
heart["famhist"] = heart["famhist"].map({"Absent": 1, "Present": 2})
X = heart.drop(columns="chd").to_numpy()
y = heart["chd"].to_numpy()
n = len(y)

rng = np.random.default_rng(4329)
fold = rng.permutation(np.arange(n) % 5)          # 93, 93, 92, 92, 92 observations
print("fold sizes:", np.bincount(fold).tolist(), " share with chd = 1:", round(y.mean(), 3))

candidates = (
    [("linear", dict(kernel="linear", C=c)) for c in [0.01, 0.1, 1, 10]]
    + [(f"poly d={d}", dict(kernel="poly", degree=d, gamma=1/9, coef0=1, C=c)) for d in [2, 3, 4] for c in [0.01,
    + [("radial", dict(kernel="rbf", gamma=g, C=c)) for g in [0.01, 0.1] for c in [0.1, 1, 10]]
)
assert len(candidates) <= 20

rows = []
for name, par in candidates:
    err = []
    for f in range(5):
        tr, te = fold != f, fold == f
        fit = make_pipeline(StandardScaler(), SVC(**par)).fit(X[tr], y[tr])
        err.append(np.mean(fit.predict(X[te]) != y[te]))
    rows.append({"kernel": name, "C": par["C"], "gamma": par.get("gamma", "–"),
                 "cv_error": np.mean(err), "se": np.std(err, ddof=1) / np.sqrt(5)})
res = pd.DataFrame(rows)
res["gamma"] = res["gamma"].map(lambda g: g if g == "–" else round(g, 3))
print(res.round(4).to_string())
```

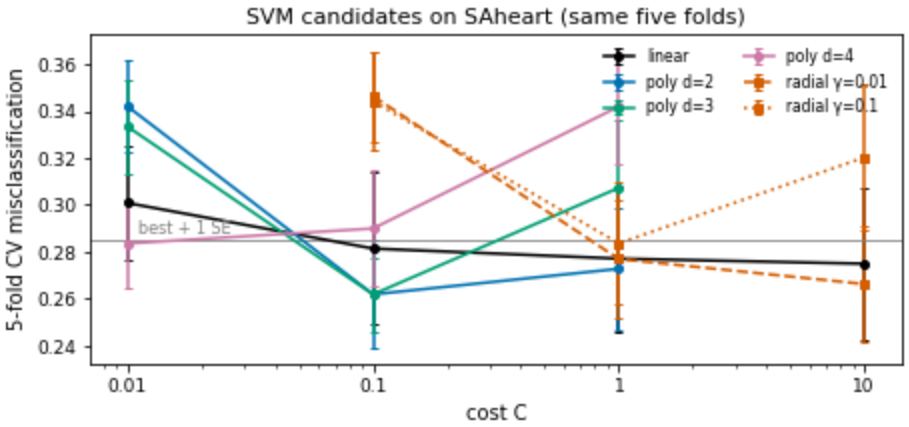
```
fold sizes: [93, 93, 92, 92, 92]  share with chd = 1: 0.346
      kernel      C  gamma  cv_error      se
0     linear   0.01      -    0.3008  0.0245
1     linear   0.10      -    0.2814  0.0325
2     linear   1.00      -    0.2771  0.0310
3     linear  10.00      -    0.2750  0.0325
4   poly d=2   0.01  0.111    0.3420  0.0196
5   poly d=2   0.10  0.111    0.2619  0.0231
6   poly d=2   1.00  0.111    0.2728  0.0257
7   poly d=3   0.01  0.111    0.3334  0.0200
8   poly d=3   0.10  0.111    0.2619  0.0158
9   poly d=3   1.00  0.111    0.3073  0.0286
10  poly d=4   0.01  0.111    0.2836  0.0188
11  poly d=4   0.10  0.111    0.2900  0.0249
12  poly d=4   1.00  0.111    0.3419  0.0246
13    radial   0.10   0.01    0.3464  0.0191
14    radial   1.00   0.01    0.2770  0.0252
15    radial  10.00   0.01    0.2663  0.0248
16    radial   0.10    0.1    0.3442  0.0208
17    radial   1.00    0.1    0.2836  0.0260
18    radial  10.00    0.1    0.3203  0.0312
```

In [2]:

```
best = res["cv_error"].idxmin()
cut = res.loc[best, "cv_error"] + res.loc[best, "se"]
print(f"best: {res.loc[best, 'kernel']} C={res.loc[best, 'C']}  CV error {res.loc[best, 'cv_error']:.4f} (SE {res.
print(f"one-SE cutoff: {cut:.4f}")
print("linear candidates within one SE:", res.query("kernel == 'linear' and cv_error <= @cut")["C"].tolist())

colors = {"linear": "#000000", "poly d=2": "#0072B2", "poly d=3": "#009E73", "poly d=4": "#CC79A7"}
fig, ax = plt.subplots(figsize=(6.5, 3.2))
for k, col in colors.items():
    s = res[res["kernel"] == k]
    ax.errorbar(s["C"], s["cv_error"], yerr=s["se"], marker="o", ms=4, capsize=2, color=col, label=k)
for g, ls in [(0.01, "--"), (0.1, ":")]:
    s = res[(res["kernel"] == "radial") & (res["gamma"] == g)]
    ax.errorbar(s["C"], s["cv_error"], yerr=s["se"], marker="s", ms=4, capsize=2, color="#D55E00", ls=ls, label=f"
ax.axhline(cut, color="gray", lw=0.8)
ax.text(0.011, cut + 0.003, "best + 1 SE", fontsize=8, color="gray")
ax.set_xscale("log"); ax.set_xticks([0.01, 0.1, 1, 10]); ax.set_xticklabels(["0.01", "0.1", "1", "10"])
ax.set_xlabel("cost C", fontsize=10); ax.set_ylabel("5-fold CV misclassification", fontsize=10)
ax.set_title("SVM candidates on SAheart (same five folds)", fontsize=11)
ax.tick_params(labelsize=9); ax.legend(fontsize=7.5, frameon=False, ncol=2)
fig.tight_layout(); plt.show()
```

```
best: poly d=2 C=0.1  CV error 0.2619 (SE 0.0231)
one-SE cutoff: 0.2850
linear candidates within one SE: [0.1, 1.0, 10.0]
```



The lowest mean CV error is 0.262, shared by the quadratic and cubic kernels at $C = 0.1$, with the radial kernel at $\gamma = 0.01, C = 10$ right behind at 0.266. None of those wins by much, though. The fold-to-fold SE of the best candidate is about 0.02, and all three linear candidates with $C \geq 0.1$ (0.281, 0.277 and 0.275) are inside one SE of the best. The curves also show what the extra flexibility costs: as C grows the polynomial and radial errors climb back up, degree 4 is already at 0.342 by $C = 1$ and the radial kernel with $\gamma = 0.1$ reaches 0.320 at $C = 10$, while the linear kernel stays flat once $C \geq 0.1$. At the small end, the quadratic and cubic kernels at $C = 0.01$ and both radial kernels at $C = 0.1$ sit near 0.34, which is the error of predicting no CHD for everyone (34.6% of the sample has CHD).

I prefer the **radial kernel with $\gamma = 0.01$ and $C = 10$** (CV error 0.266, SE 0.025). It is 0.004 behind the quadratic and cubic kernels at $C = 0.1$, which is far less than one SE, and about

one point better than the best linear setting (0.275). What decides it for me is how each kernel behaves around its best setting. The cubic kernel goes from 0.262 at $C = 0.1$ to 0.307 at $C = 1$, and the quadratic kernel sits at 0.342 one step below its best, so a small change in C moves them a lot. The radial kernel with $\gamma = 0.01$ improves steadily from 0.346 to 0.277 to 0.266 as C grows, and its larger-γ neighbour is still at 0.284 at $C = 1$, so the result is less sensitive to the exact setting. With $\gamma = 0.01$ on standardized predictors the kernel is very smooth, so the boundary is close to linear with only mild curvature. The honest caveat is that all of the top settings, linear included, are within one SE of each other, so the one-standard-error rule would pick the linear kernel at $C = 0.1$. The radial choice is a small bet on curvature that these five folds can neither confirm nor rule out.

Question 2: A short SVM analysis skill

The skill. My skill is `.claude/skills/svm-report/SKILL.md` in my course repository (the folder is git-ignored, so the complete file is appended after the report). The frontmatter sets `disable-model-invocation: true`, so Claude only loads it when I type `/svm-report` myself. The body is 27 lines, one numbered step for each of the seven things the skill has to direct, and it names no dataset, variable or result.

The reviews from both sides shaped most of it. The review of my Homework 7 skill pointed out that my selection rule broke ties only within 0.001 while another rule told the agent to admit when the winner was inside the fold noise, so the new skill selects with a one-standard-error rule and spells out the simplicity order (linear, then polynomial by degree, then radial, then smallest C). The same review said my skill only worked through `$ARGUMENTS`, never said where the report goes and never checked the page limit, so this one accepts the inputs either way, asks for one HTML file plus the script, and makes the agent print the report and cut words until it fits on one page. It also said my figure crammed 21 labeled points on one axis, so the figure here is one line per kernel against C. From the review I wrote of Lyndsey's skill I kept two things I had asked her for, an SE next to every CV error and a confusion matrix with the error in each class.

Applying it. I invoked the skill in a fresh Claude Code agent that was given only the skill path and these details: data `homework/week-09/data/saheart.csv`, response `chd`, the nine other columns as predictors with `famhist` coded 1 for `Absent` and 2 for `Present`, seed 4329, and an output folder `homework/week-09/skill-run/`. The agent wrote `svm_report.py` and `report.html` there. The report below is that file as produced, except that I raised its table font from 10 to 11 points to meet the 11-point requirement and reran the script.

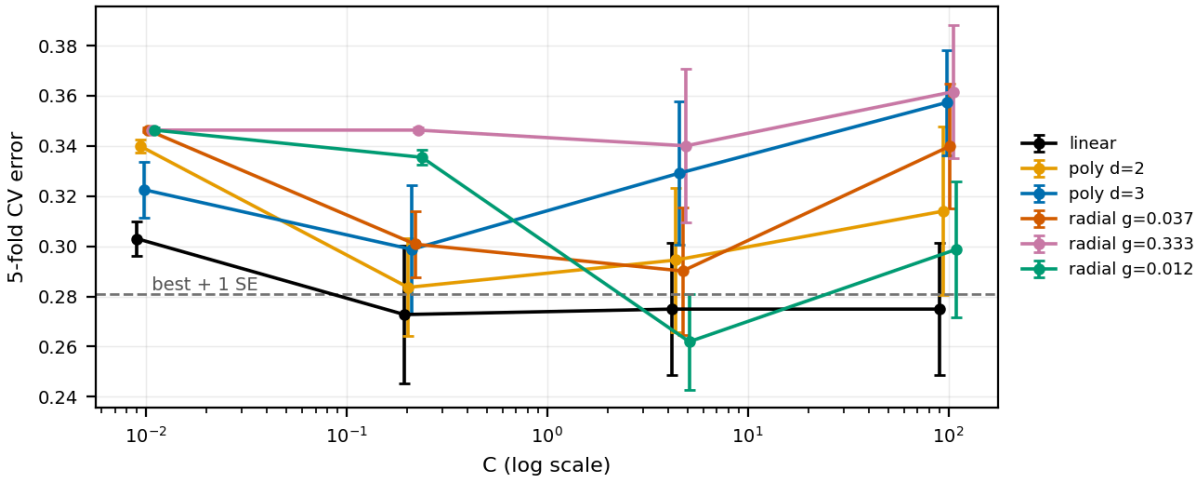
SVM classification of chd, SAheart data (n = 462, p = 9, seed 4329)

Data check

462 rows, 0 missing values. Classes: chd=0 302, chd=1 160. The smaller class is 34.6% of rows, so no class is under 10%; n is large next to p = 9. Eight predictors are numeric as given; famhist (Absent/Present) is coded 1 = Absent, 2 = Present.

Method

Linear, polynomial (degree 2 and 3, coef0 = 1, gamma = 1/p) and radial SVMs (scikit-learn SVC, default tolerance, no class weights) were scored on the same 5 stratified folds (seed 4329), standardizing inside each fold on the fitting rows only. C ran over 0.01, 0.215, 4.64, 100 and radial gamma over 0.037, 0.333 (1/p = 0.111), 20 settings. Edge check: radial: best gamma=0.037 on edge, added gamma=0.012 over the C grid, 24 settings in all. The radial best is again on the gamma edge (0.012); the grid was widened only once. The choice uses the one-SE rule (linear, then polynomial by degree, then radial; smallest C first).



Kernel	Best setting	CV error	SE	Support vectors
linear	**C=0.215**	**0.273**	**0.028**	**282**
poly d=2	C=0.215, coef0=1, gamma=1/p	0.284	0.019	283
poly d=3	C=0.215, coef0=1, gamma=1/p	0.299	0.026	276
radial	C=4.64, gamma=0.012	0.262	0.019	284

Selected model: linear, C=0.215

Lowest CV error 0.262 (SE 0.019, radial g=0.012, C=4.64), so the cutoff is 0.281. 4 settings fall within it and the simplest is linear, C=0.215 with CV error 0.273; refit on all rows it keeps 282 support vectors.

CV confusion	pred 0	pred 1	class error
true 0	253	49	0.162
true 1	77	83	0.481

Findings

No curved kernel beats linear by more than the noise: the best curved fit (radial, 0.262) is 0.011 below linear's 0.273, against a linear SE of 0.028. Raising C narrows the margin: the linear SVM keeps 313 support vectors at C=0.01 and 278 at C=100 (refit on all 462 rows). The selected model misses 16.2% of the no-CHD cases and 48.1% of the CHD cases in cross-validation, so it errs mostly on the smaller class. Largest standardized coefficients: famhist +0.46, age +0.41, typea +0.36, tobacco +0.34.

Limitations

Every setting was scored on the same folds and the best was picked from them, so the best CV error is optimistic. There is no separate test set.

Checking the report. I recomputed the selected model outside the agent's script with the same stratified folds (`StratifiedKFold` , shuffle, seed 4329). The linear SVM with $C = 0.215$ gives CV error 0.273 with SE 0.028, the radial setting with $\gamma = 0.012, C = 4.64$ gives 0.262 with SE 0.019, the out-of-fold confusion matrix is 253/49 and 77/83, and the refit keeps 282 support vectors, so every number in the report matches. The one-SE logic is also right, since 0.273 is under the cutoff 0.262 + 0.019 = 0.281 and linear comes first in the order. The run exposed two weak spots in my skill. The grid step asks for 20 settings or fewer and also for widening at an edge, so the agent went to 24 settings, and its radial best was still on the small-γ edge after one widening. The skill should say whether widening counts against the cap. The report and Question 1 agree on the main point, that no curved kernel beats linear by more than one SE on this data. The skill's rule picks linear, while in Question 1 I chose the radial kernel on purpose as a small bet on curvature.

Appended `SKILL.md`

```
../../.claude/skills/svm-report/SKILL.md: instruction body 28 lines (limit 50)
-------------------------------------------------------------------------------------------------------
---
name: svm-report
description: Fit and tune SVM classifiers on a dataset the user describes and write a one-page report. Only runs w
hen invoked explicitly with /svm-report.
disable-model-invocation: true
---

The user gives the data file, the response, the predictors and their coding, and a seed, either as $ARGUMENTS or i
n the message.
Ask once if one is missing. Do not assume anything else about the data.

1. Check the data first. Report n, the class counts, missing values and the predictor types. An SVM needs a
   two-class (or few-class) response and numeric predictors, so code categories as the user says. If one class is
   under about 10% or n is small next to p, say so in the report.
2. Kernels. Always fit a linear kernel as the baseline. Add polynomial (degree 2 and 3, coef0 = 1) and radial
   kernels so the report can say whether a curved boundary helps. Write down every setting held fixed.
3. Tuning. Tune C for every kernel and gamma for the radial kernel on log grids (C from 0.01 to 100, gamma
   around 1/p). Keep the whole grid at 20 settings or fewer. If the best setting is on the edge of its grid,
   widen the grid once and say that you did.
4. Evaluation. Make K = 5 folds with the seed, stratified by class, and use the same folds for every setting.
   Standardize inside each fold using only the fitting rows. Score the mean misclassification rate and its
   SE (sd / sqrt(K)). Choose with the one-SE rule: among settings within one SE of the best, take the simplest,
   in the order linear, then polynomial by degree, then radial, and the smallest C within a kernel.
   Refit the choice on all rows and report its CV confusion matrix and the error within each class.
5. Figure and table. One figure about 6.5 x 2.6 in: CV error against C on a log axis, one line per kernel
   (per gamma for radial), SE bars, and a horizontal line at best + 1 SE. Okabe-Ito colors.
   One table with the best setting of each kernel: kernel, settings, CV error, SE, number of support vectors.
6. Interpretation. Say whether the curved kernels beat linear by more than the noise, what C did to the
   margin (support vectors), and how the selected model does on each class. For a linear choice, list the
   largest standardized coefficients. Every number must come from output printed in this run.
7. Report. Write one self-contained HTML file plus the script that made it. Use 11-point font, letter size.
   Order: title line (data, n, seed), data check, method in two or three sentences, figure, table, selected
   model with its confusion matrix, findings, and limitations (same folds for all settings, so the best CV
   error is optimistic; no separate test set unless the user gives one). Print it to PDF and check it is one
   page; if not, cut words first, then shrink the figure. Never drop the table, figure or the chosen numbers.
```